1.0 Symmetric vs. Asymmetric Key Encryption

Symmetric Key Encryption uses a single key for both encryption and decryption. Its essential requirement is that the key must be securely shared between sender and receiver before any secure communication can take place. If the key is intercepted, the entire confidentiality of the communication is compromised.

Asymmetric Key Encryption, on the other hand, uses a key pair: a public key (for encryption) and a private key (for decryption). The public key can be shared openly, and only the corresponding private key can decrypt the message. This eliminates the need for secure key exchange but typically operates slower than symmetric encryption.

Symmetric Key Encryption	**Asymmetric Key Encryption**
Involves a single key for both encryption and decryption.	Uses a pair of keys: a public key (for encryption) and a private key (for decryption).
Essential requirement: Both sender and receiver must securely share and keep the same secret key.	Public keys can be shared openly, while the private key is kept secret.
Examples: AES, DES, RC4.	Example: RSA, ECC.

2.0 Public-Key Cryptosystem Requirements

A secure public-key cryptosystem must fulfill the following:

1. Computationally Easy Key Generation:
    - Generating key pairs (public & private) must be efficient.

2. Hard to Derive Private Key from Public Key:
    - It should be computationally infeasible to determine the private key from the public key (e.g., based on the RSA factoring problem).
3. Encrypting with Public Key and Decrypting with Private Key Must Work Reliably:
    - The system must correctly and consistently decrypt messages encrypted with the corresponding key.
4. Resilience to Chosen-Ciphertext and Chosen-Plaintext Attacks:
    - Must resist common cryptographic attacks.

3.0 AES Transformations:

(a) Sub-Bytes:

- A non-linear substitution using an S-box.
- Each byte is replaced independently to introduce non-linearity and confusion.
- The S-box is designed to resist linear and differential cryptanalysis.

(b) Shift-Rows:

- Row-wise shifting of the AES state matrix:
    - Row 0: No shift.
    - Row 1: Left shift by 1 byte.
    - Row 2: Left shift by 2 bytes.
    - Row 3: Left shift by 3 bytes.
- Purpose: Spreads out (diffuses) the bytes over multiple columns, making patterns harder to detect.

4.0 Hash Functions, Blockchain, RSA, PoW

a) Cryptographic Hash Function

A hash function is a mapping:
H: {0,1}* → {0,1}^{n}

Significance in blockchain:

- Ensures immutability: any change in data changes the hash.
- Guarantees integrity: each block contains the hash of the previous, making tampering evident.

(b) Hash Calculation and Tampering Effect

Given:

- D_1 = "Vote123"
- H_0 = "abcXYZ"
- H_1 = SHA256("Vote123abcXYZ") = 9f6a2...

If D_1 is tampered, H_1 will change. This breaks the chain as H_2 will now refer to a modified H_1, triggering integrity violation across the blockchain.

(c) RSA Digital Signature

Given:

- H(M) = 13
- d = 7 (private key exponent)
- n = 33
- e = 3 (public key exponent)

i) **Compute Signature**:
$S = 13^7 \bmod 33$

$13^7 = 62748517$
$62748517 \bmod 33 = 25$

ii) Verify Signature:
Verify if $25^3 \bmod 33 = 13$
$25^3 = 15625$
$15625 \bmod 33 = 13$

So it proves that the Signature is valid.

(d) Proof-of-Work Check
Formula:

$$SHA256(N \ // \ BlockData) < T$$

Where:

- N = Nonce (random number miners adjust to find a valid block)
- BlockData = transaction data, previous hash, etc.
- T = 240 = Difficulty threshold (a small number = harder to satisfy)

Here:

SHA256("005XXXXX") = 3239
So compare: 3239 > 240 → Does NOT satisfy PoW condition

What This Means:

- The nonce "005" doesn't make the hash small enough.
- To "mine" a block, miners must keep trying new nonce values until they find one where SHA256(N // data) < T.
- This is computationally expensive, which is why it secures blockchains like Bitcoin.